Lemma Boundaries Improve Morphological Alignment of Subword Tokenizers but Worsen Language-Model Loss

Jessie Dong*
Stanford University
jessiedong@stanford.edu

Anish Chaudhuri*
Stanford University
anishch@stanford.edu

Maya Vendhan*
Stanford University
mvendhan@stanford.edu

Abstract

The prevalent byte pair encoding (BPE) model and the unigram model use only the characters in a text to build a vocabulary. The cheapest kinds of grammatical information are information about the part-of-speech of each word, and lemmas of each word, which can be provided by a standard part-of-speech tagger. A controlled evaluation compares these two models with tokenizers that use part-of-speech and lemma information in five languages: English, Spanish, Danish, Tamil, and Turkish. The same Wikipedia text and 8,000 type budget are used for all tokenizers. The first version of a BPE tokenizer uses the probability of the part-of-speech transition at each word boundary in the scoring function for merge operations. The second version uses part-of-speech weighting of merges within a content word (noun, verb, or adjective) so that content words stay whole. Finally the Lemma2Word tokenizer uses the lemma to determine where the boundary between stem and affixes should be.

We trained each model on three different samples of text, evaluated the model on its ability to find morpheme boundaries (the gold standard is the lemma-derived boundaries of MorphScore), held-out compression, stability of the model, and bits per character of a small language model trained on the model's output. The part-of-speech weighting of transition probabilities has no effect, in four out of five languages, on the F1 score for morpheme boundary detection (the difference is less than the variation from seed to seed), and of course does not affect held-out compression. This is not surprising, since these affect tokens that are common function word pairs, and of course a part-of-speech transition probability does not convey information about the interior of a word. However the part-of-speech weighting of the merger of tokens within content words raises the F1 score for finding morpheme boundaries from 4.4% to 22.2% in Tamil, while lowering the scores in Spanish and Turkish. Lemma2Word raises boundary recall in every language by 23 to 77 points (with 4–8% more tokens). When the tokenizer does not see the word in its training text, the model retains most of the gain if the language's affixes are long enough for an inventory of them to generalize.

The probe with a language model ranks the tokenizers differently. For example, the unigram model has the lowest number of bits per character in every language, while Lemma2Word has the highest. This shows that a tokenizer that finds more morpheme boundaries is not necessarily a better choice for a language model, at least at the scale tested here.

1 Introduction

The subword tokenizer determines the string unit on which a language model operates. Common tokenizers like byte-pair encoding [23] and the unigram language model [13] learn the boundaries of unit tokens from the statistical co-occurrence of characters alone. It is for this reason that they

*Equal contribution.

can never learn that *walking* is composed of *walk* and *ing*, that *the* usually precedes nouns and adjectives, or that Turkish *evlerimizden* happily deposits a cascade of four suffixes at the end of *ev*. In recent years there has been a growing effort to incorporate additional morphological knowledge into tokenizers [4, 10, 12], and in this work we explore the capabilities of adding syntactic knowledge. POS taggers for more than 60 languages are widely available [20], and tagging a corpus is a one-shot process. POS tags provide a reusable signal about the grammar of a text and are unified across languages under the Universal Dependencies scheme [17].

The three tokenizers we evaluate use the POS tags from the tagger as a signal and their lemmas to guide training on a BPE vocabulary. The first method we describe weighs the BPE merge probability based on the likelihood of a POS transition across word boundaries. The second weights the probability of merges inside content words so that nouns, verbs, and adjectives stay whole. The third, called Lemma2Word, uses the lemma from a POS tagger to produce a boundary indicating the stem of a word which it then segments before subword training. We also evaluate a large language model prompted to segment words, as a segmenter rather than a tokenizer.

Initial tests without held-out words, gold boundaries, or repeated samples showed promising results on the core tokenizer evaluation metrics (segmentation entropy, token length, and vocabulary stability) but did not directly address the use of a tokenizer. In the controlled evaluation that follows, we provide that evidence for 5 languages: English, Spanish, Danish, Tamil, and Turkish. Methods were trained with the same Wikipedia text under an 8,000-type budget and each was repeated three times using different samples to train the tokenizers. We compared all methods to both BPE and the unigram model across a series of metrics: boundary precision and recall against gold boundaries produced by MorphScore [3] (derived from treebank lemmas), compression and stability on held-out text, and bits-per-character for a small language model trained on the output of each tokenizer.

The first method that weights BPE merges based on POS transition probability hardly affects the results, since it acts only on tokens that absorb a space at their ends. It produces the same pairs of frequent function words as unweighted BPE and changes the boundary F1 less than the variance between seed-to-seed runs for four of the five languages. The second method that weights merges inside content words affects each language differently. The improvement in Tamil is substantial but it is actually a negative effect for Spanish and Turkish. The third method, Lemma2Word, increases boundary recall by 23–34 points for four languages and by 77 points in Tamil in a way that adds 4–8% more tokens. Across English, Tamil, and Turkish, Lemma2Word retains most of this gain even on words its lexicon has never seen.

Finally, we conclude that the unigram language model remains the best-performing tokenizer that makes no use of linguistic knowledge and is the only one competitive in Tamil without a lemmatizer. The language model probe does not follow the boundary scores and in all five languages ranks the unigram model as first and Lemma2Word as last in bits per character. In other words, finding more morpheme boundaries does not guarantee a better tokenizer for a language model, at least at the 7M scale we tested. All code, data preparation scripts, and numbers are produced by the released repository.

2 Related Work

Both byte-pair encoding [23] and the unigram language model [13], popularized in the SentencePiece implementation [14], are subword tokenizers that learn token boundaries only from the statistics of characters. Gallé [9] attributes the success of BPE to the shorter sequences it produces. Mielke et al. [16] discuss the space of choices between characters and words, and Bostrom and Durrett

[6] observed that, in English, unigram segmentations are closer to morphemes and that language models trained on them were at least as good. We reproduce that finding here for five more languages.

Unsupervised morphology induction dates back decades before the advent of neural language models (Morfessor [8] being a prime example) and more recently there has been effort to leverage morphological information in BPE or WordPiece vocabularies. MorphPiece [12] injects morphological information by using morphological dictionaries with BPE, MorphBPE [4] constrains BPE merges based on morpheme boundaries produced by a morphological analyzer, MorphTok [7] applies morphological segmentation to Indic text before subword segmentation, and KinyaBERT [18] constructs a morphology-aware encoder for Kinyarwanda. Hofmann et al. [10] show that segmenting words at derivational boundaries improves BERT's comprehension of complex words, and Hofmann et al. [11] obtain a similar effect by choosing the segmentation with the lowest number of tokens from a fixed vocabulary. Lemma2Word falls into this category, though instead of choosing a morphological segmentation from a dictionary, we derive boundaries from a lemmatizer.

In the case of unspaced languages it has been common practice to perform segmentation and POS tagging jointly rather than sequentially, with improvements in performance [27]. We are not aware of any prior work on subword tokenization that makes use of POS information to weight merge operations, and the POS-weighted BPE method evaluated herein is new in that sense, which we use to assess what this methodology is capable of, as well as what it is not.

Evaluation of morphological alignment with word segmentation lexica uses precision and recall metrics [3, 5], and weak alignment has been linked to poorer language modeling performance for morphologically complex languages [2]. The gold standard referenced here is MorphScore [3], which is computed from word boundaries derived using Universal Dependencies lemmas for 70 languages. Intrinsic metrics like compression and Rényi efficiency [28] are imperfect proxies for performance in downstream tasks. As observed by Schmidt et al. [22], the number of tokens is not a very good metric of progress. Ali et al. [1] report that tokenizer choice is measurable at the 2.6B model scale. Rust et al. [21] found that monolingual tokenizers beat a single shared multilingual tokenizer. Park et al. [19] used morphology to predict language-modeling difficulty on 92 languages.

Toraman et al. [24] evaluated tokenizers specifically for Turkish, one of the languages considered here; they find that morphology-level tokenization is useful for small vocabularies. Our language-model probe follows the practice of these studies, but on a much more modest scale.

3 Tokenizers Under Test

All of these tokenizers ingest the same text, and produce an 8,000-type vocabulary for each language. They all tokenize on whitespace, but they differ in what additional information they draw on. Two of them, the baselines, do not use any additional information, and three of them, the POS-weighted variants, receive POS tags for every word of the training set. The Lemma2Word tokenizer, in addition, receives a lemma for every word of the training set.

3.1 Byte-Pair Encoding with a Free Space Marker

The byte-pair encoding (BPE) algorithm [23] constructs a vocabulary by repeatedly merging the most frequent symbol pair. Our implementation of BPE inserts a marker (␣) between words. This marker can be merged with the word that precedes it (`the␣`) or with the word that follows, or become internal to a token that spans two words (`of␣the␣`). In SentencePiece [14], ␣ is always prefixed to each word at the start of training, so that tokens can never span word boundaries.

In this evaluation, we distinguish between the former, the free-marker variant (*BPE, free marker*), which is the one that we will use to demonstrate POS-weighted tokenization, and the latter, the word-initial marker variant (*BPE, word-initial marker*). Vocabularies are made up of the alphabet of the text supplemented with the learned merge operations.

3.2 Part-of-Speech-Weighted Merges

Universal Dependencies POS tags are associated with every word of the training set, and a transition probability table $P(b \mid a)$ is computed from this tagged text. The POS-weighted variant replaces the pair count with a sum weighted over its occurrences. The weight of joining the end of a word with the marker following it, which we are interested in, is $1+p$, where p is the transition probability between the POS tags of those words, clamped between 0.01 and 0.95.

If the marker is bare on either side, the weight is further multiplied by 0.98, and all other cases have a weight of 1. This scheme encourages tying a trailing space to its word when the next word is predictable from its tag, slightly discourages interaction with the marker, and has no impact on internal pairs. A first ablation is to drop the 0.98 multiplier, which we term *transitions only*. A second is to drop the transition term, which we call *penalty only*. In the earlier iteration of this implementation, the scores were recomputed every time they were needed, so they could only be computed for small vocabularies. The updated implementation maintains a list of all occurrences for each pair, and updates their scores after each merge, which allows training of an 8,000-type vocabulary on 3,000 paragraphs in less than a minute.

There are two defects in the earlier implementation. First, transitions that had never been observed were assigned a probability of 0.6, which is more than most of the observed values; this has been changed to the floor value of 0.01. Second, in the older version, the word to which each symbol belonged was recovered by searching for bare markers in the current symbol sequence, so that when a marker was merged it was no longer counted, and later computed weights of transitions applied to a different pair of words. In the updated version, the word for each symbol is recorded when the symbol sequence is built. If both of these fixes are disabled, the rewritten code computes the same pair scores before the first merge, and the same unweighted merge sequence, as the original, as can be confirmed in the tests in `experiments/test_posbpe.py`.

3.3 Content-Word Weighting

The goal of the POS signal is to preserve content words (such as *scientist*) intact rather than splitting them into components. The merge weighting method in Section 3.2 does not achieve this goal because it never changes the score of a pair inside a word. The simplicity of the approach led us to propose a variant that weights all pairs within words having POS tags NOUN, PROPN, VERB, and ADJ by two and all other pairs by one. The weight of two was selected before running the experiments and never tuned. This variant moves vocabulary capacity toward whole content words and away from function words and affixes.

3.4 Lemma2Word

To develop Lemma2Word, we first obtain the Stanza lemma of each word occurring in the training text. The stem is the lemma if it is a substring of the word (*walk* in *walking*); otherwise we take the longest common substring of length at least three, such as *habl* for the words *hablo* and *hablar*. Anything before the stem is a prefix and anything after is a suffix; at most two boundaries can occur in a word. We derive three items from the training text. The first item is a lexicon that

maps each word to its boundaries, using majority vote to resolve inconsistent lemmatization. The other two are the inventory of suffixes and of prefixes, each being counted.

A word-form occurring in the lexicon is segmented according to it. A word not occurring in the lexicon is first bounded by the longest suffix such that the resulting stem has at least three characters; then it is bounded from the left by the longest prefix such that the resulting stem has at least three characters, if the suffix and prefix have each been seen at least 20 times. Finally we encode the subword units, using a free marker BPE tokenizer trained on the pre-segmented text so as to not enable a merge crossing a Lemma2Word boundary. The variant called *lexicon* does not use the lemmatizer to obtain the lexical units, but does use it to construct the lexicon.

The *online* variant uses Stanza's lemmatizer to obtain the lemma of the word being tokenized, and then uses the lemma to obtain the stem, as above; if the word and the lemma have no common substring, we fall back to the lexicon rule. This online variant is the same as in the original report. Unlike the lexicon variant, the online variant requires a tagger at tokenization time, and it shares training data with the gold standard, as shown in Section 4.3.

3.5 Reference Tokenizers

We implemented two reference tokenizers. One is a BPE tokenizer with a word-initial marker and the Metaspace pre-tokenizer, as implemented in the Hugging Face `tokenizers` library. The other is the unigram model tokenizer implemented in SentencePiece [13], which uses all characters and does not perform Unicode normalization, which ensures that all tokenizers have the same character set.

3.6 Using a Language Model as a Segmenter

In the original report, we explored the possibility of utilizing a large language model to segment words. We assess it as a segmenter rather than a tokenizer, since we provide the model with a fixed set of words to evaluate and the prompt instructs the model to mark morpheme boundaries without modifying any characters. These boundaries are subsequently assessed in the same way as other methods. The findings of this study are discussed in Section 5.8, and the prompt is detailed in Appendix C.

4 Evaluation Protocol

4.1 Languages and Text

Five languages were chosen for the original report: English, Spanish, Danish, Tamil, and Turkish. English, Spanish, and Danish are Indo-European, and Tamil and Turkish are Dravidian and Turkic respectively, so the five cover three families and two scripts. English is predominantly isolating, while Tamil and Turkish are agglutinative. Spanish and Danish have properties situated between these two extremes.

The training and the held-out data come from the same source. Specifically, we downloaded a snapshot of Wikipedia [26] dated 2023-11-01. For each language, we skip the first 2,000 articles, which are usually short administrative pages, and take the paragraphs from the next 6,000 articles with length between 200 and 1,500 characters.

After shuffling the paragraphs with a fixed seed, we allocated 4,000 to the training pool and 500 to the held-out set. A training seed picks 3,000 paragraphs from the training pool. Thus, three training seeds for a method will differ in both their training data and the random behavior of the

method itself. This variation is a proxy of the sensitivity to the data sample. The training sample contains from 143,704 words (for Tamil) to 246,661 words (for English).

4.2 Tags and Lemmas

Stanza version 1.11 [20] tags and lemmatizes all paragraphs from the training set and the held-out set. However, the tokenization of Stanza does not match the tokens delimited by whitespace because of the separation of punctuation and because some languages (Spanish and Turkish) have multi-word tokens. Thus, each token delimited by whitespace is assigned the tag and the lemma of the first Stanza word with no punctuation that starts within it; if there is no such word, we use the tag PUNCT. The transition table of Section 3.2 and the lexicon of Lemma2Word are computed on the training words aligned with Stanza for each seed.

4.3 Gold Morpheme Boundaries

Boundary accuracy is measured using MorphScore version 2 [3], which provides segmentations for all word forms in a Universal Dependencies treebank [17]. These are derived from the lemmas of the words, with the stem being the part of the word form that is also in the lemma, and affixes being the material before and after the stem. Each form has a maximum of 2 boundaries, and no internal derivational structure is indicated (*un* | *believ* | *able*). Thus a tokenizer that creates boundaries within the stem is penalized for precision, even if this is linguistically appropriate. Recall is measured to capture whether the tokenizer segments inflectional affixes from the stem, and is considered the main metric.

As with the default of the MorphScore release, we keep only unique forms where the lemma is equal to the recorded stem, discarding forms containing digits or punctuation, or having parts that do not reassemble into the form. This leaves a set of 4,089 (English), 6,712 (Spanish), 6,650 (Danish), 1,193 (Tamil) and 30,192 (Turkish) forms from the EWT, AnCora, DDT, TTB and Kenet treebanks. Each word is tokenized on its own, markers are removed, and character offsets of the token boundaries are compared to gold. Precision, recall and F1 are micro-averaged over all boundaries.

We will report two subsets in addition to the entire set. A subset of forms unseen by the tokenizer have not been seen in the training data, so finding a boundary is not possible using the dictionary of Lemma2Word forms, and these must be produced from internal word-structure analysis. The UD test subset is from the test split of the treebank, and is significant for the online version of Lemma2Word. Note that while Stanza's lemmatizers are trained on the training splits of the same treebanks used by MorphScore and so can remember the gold lemma for training split forms, the tagger trained on the treebank cannot influence other tokenizers that only learn from the Wikipedia text.

4.4 Compression and Stability

All tokenizers are scored for every held-out paragraph in terms of number of tokens per whitespace word, and characters per token (markers not counted). We also measure what fraction of tokens cross a word boundary, and, for free-marker tokenizers, look up the POS transition at each crossed boundary in the training transition table, to check if POS weighting changes which boundaries are crossed. Additionally we measure stability over three seed pairs, as the Jaccard overlap of vocabularies, and the boundary F1 between two segmentations of the same held-out text.

4.5 Language-Model Probe

The utility of the tokenizer is directly probed at a small scale by training a 4-layer transformer language model [25] for each tokenizer with 256-dimensional token embeddings, 4 attention heads and a context window of 128 tokens (7.3M parameters). Each model is trained for 6 passes over the tokenized training paragraphs, using AdamW optimization [15] with learning rate 3×10^{-4} with warm-up and cosine decay, and a batch size of 64 sequences. The model is scored on held-out paragraphs using non-overlapping context windows, and the total negative log-likelihood (bits) is divided by the number of characters in the held-out text.

As this denominator is the same for all tokenizers of a given language, we can directly compare tokenizers that produce different numbers of tokens in terms of bits per character. Each tokenizer seed results in one language model, with a fixed model seed in order to provide a fair comparison across seeds.

4.6 Compute

All tokenizers and probes are trained on a single laptop. Each free-marker BPE or its weighted variants on 3,000 paragraphs takes about a minute on 1 core, and the reference tokenizers only seconds. Each language-model probe takes 1–3 minutes on the laptop's GPU, and the full experiment can be repeated in a few hours with 120 tokenizer runs, 120 probes, and a vocabulary sweep.

5 Results

5.1 Boundary Accuracy

Table 1 and Figure 1 show boundary F1 scores compared to the gold derived from MorphScore's segmentation into morphemes based on lemmas, for all methods and languages. Precision and recall scores are in Tables 7 and 8 of the Appendix, and all methods are shown in precision-recall space in Figure 2.

Table 1: Boundary F1 (%) compared to MorphScore gold boundaries derived from lemmas, over all unique word forms. Average and standard deviation over 3 training seeds. The best result for each language is in bold.

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	40.0 ±1.6	18.7 ±0.8	24.3 ±0.8	4.4 ±2.3	35.0 ±1.9
POS-weighted BPE	43.4 ±1.6	18.9 ±0.8	24.5 ±0.5	6.2 ±2.3	35.0 ±1.6
transitions only	43.4 ±1.6	18.9 ±0.8	24.5 ±0.5	6.2 ±2.3	35.1 ±1.6
penalty only	42.0 ±1.3	18.7 ±0.8	24.4 ±1.0	4.4 ±2.3	34.9 ±1.9
Content-word-weighted BPE	43.9 ±0.2	16.5 ±0.3	27.8 ±0.7	22.2 ±1.1	34.2 ±0.4
Lemma2Word + BPE	**59.9 ±0.1**	**36.3 ±0.1**	**43.8 ±0.3**	**52.9 ±0.4**	**48.7 ±0.1**
BPE, word-initial marker	25.5 ±0.4	6.7 ±0.5	13.1 ±0.1	7.5 ±0.3	40.8 ±0.4
Unigram LM	54.3 ±0.5	29.0 ±0.0	35.1 ±0.5	43.0 ±0.8	46.5 ±0.4

The results in Table 1 also show that Lemma2Word (Section 3.4) offers the best F1 score in all languages, with a range from 36.3% in Spanish to 59.9% in English. The unigram model is in second place in all languages, and leads all BPE variants by a wide margin in Tamil (F1 of 43.0 vs. 4.4). POS-weighted BPE differs from its unweighted base by about three percentage points in English (43.4% vs. 40.0%) and by less than a point in Spanish, Danish, and Turkish. The difference

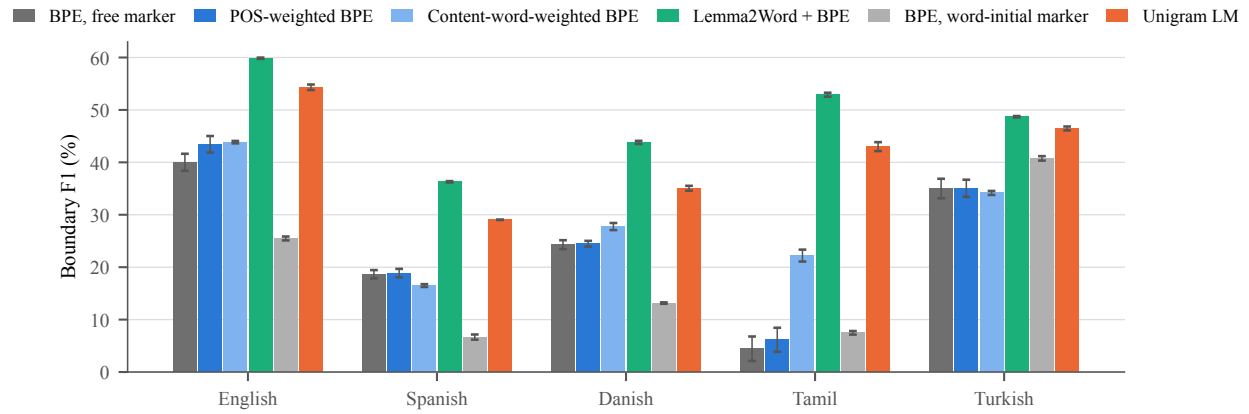


Figure 1: Boundary F1 (%) for the six main tokenizers, over 5 languages. The bars show the average over 3 training seeds, and error bars one standard deviation.

in English is the only one bigger than the variation due to random seeds. Ablation experiments also show that the transition term alone accounts for the improvement; a version using only the transition term produces the same F1 score as the full model while a version using only the penalty term performs the same as the unweighted base model.

The F1 score using content-word weighting (Section 3.3) increases over plain BPE from 4.4% to 22.2% in Tamil and by about three percentage points in English and Danish, but decreases in Spanish and Turkish. The variant with a word-initial marker (Section 3.1), following the SentencePiece standard, produces the worst scores in four languages because the whole word plus the marker is a single token and thus has fewer word boundaries inserted than the variant with a free marker; in Spanish for example its recall is only 9.1% vs. 29.4% for the free marker variant.

Figure 2 shows boundary precision versus recall for each language (points indicate the mean over the three seeds, error bars show one standard deviation, and most are smaller than the markers).

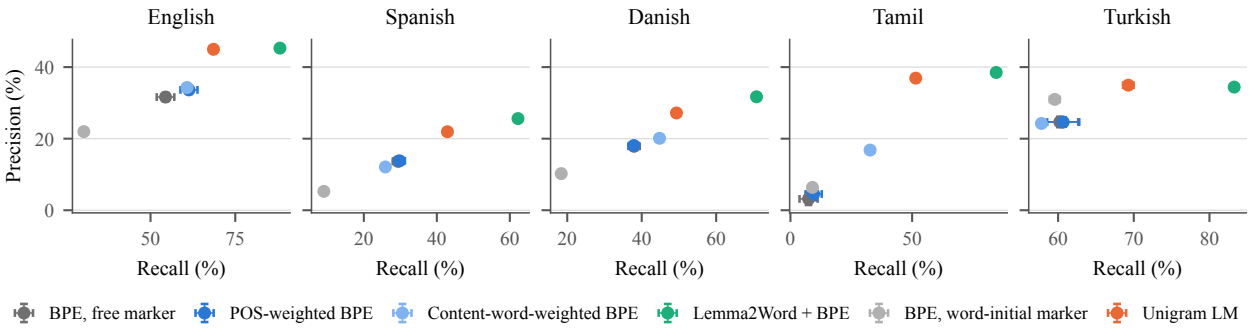


Figure 2: Boundary precision against recall per language. Points are means over three seeds and error bars span one standard deviation; most are smaller than the markers.

5.2 What the Part-of-Speech Weighting Changes

The weight in Section 3.2 affects merges only at the right word boundaries, i.e. when a word is merged with the following space. The effect of these merges is measured on held-out text in Table 2. Plain free-marker BPE puts word boundaries inside tokens in 15.5% to 54.1% of cases (for Turkish and Spanish respectively), while the POS weighting does not change this proportion much in most

languages, but reduces it from 54.1 to 39.5 in Tamil[1]. The transitions spanned are the same ones BPE spans: in English, for example, cases of an adposition before a determiner (`of␣the␣`), in Spanish cases of determiner before noun, in Danish cases of noun before adposition.

Across all languages, variants, and with or without weighting, the mean transition probability at a spanned boundary is no more than two percentage points away from the mean over all boundaries. The proportion of tokens crossing boundaries depends on how often each type of boundary occurs. The transition probabilities are no more than frequencies of tag pairs and carry no additional information beyond the counts of symbols.

Table 2: Percentage of word boundaries spanned by a token in held-out text, mean probability (%) of the POS transition at spanned boundaries, and the same mean over all boundaries. Results are from seed 0; the most common spanned transition and token are shown for the POS-weighted model.

Language	System	spanned (%)	P, spanned	P, all	top transition	top token
English	BPE, free marker	45.2	23.3	22.1	ADP→DET	`in␣the␣`
	POS-weighted BPE	42.6	23.5	22.1	ADP→DET	`of␣the␣`
	Content-word-weighted BPE	33.2	24.4	22.1	ADP→DET	`of␣the␣`
Spanish	BPE, free marker	54.1	32.3	28.2	DET→NOUN	`a␣de␣`
	POS-weighted BPE	54.1	32.3	28.2	DET→NOUN	`a␣de␣`
	Content-word-weighted BPE	43.5	33.8	28.2	DET→NOUN	`en␣la␣`
Danish	BPE, free marker	41.3	21.1	22.4	NOUN→ADP	`en␣i␣`
	POS-weighted BPE	41.4	21.1	22.4	NOUN→ADP	`en␣i␣`
	Content-word-weighted BPE	31.0	20.9	22.4	NOUN→ADP	`er␣i␣`
Tamil	BPE, free marker	54.1	24.4	24.9	NOUN→NOUN	◌ா␣␣
	POS-weighted BPE	39.5	25.0	24.9	NOUN→NOUN	◌ா␣␣
	Content-word-weighted BPE	7.4	26.5	24.9	NOUN→NOUN	◌̇␣␣
Turkish	BPE, free marker	15.5	22.3	25.0	NOUN→CCONJ	`i␣ve␣`
	POS-weighted BPE	15.2	22.5	25.0	NOUN→CCONJ	`i␣ve␣`
	Content-word-weighted BPE	9.0	23.7	25.0	NOUN→CCONJ	`i␣ve␣`

5.3 Content-Word Weighting

Doubling the weight in Section 3.3 on merges inside some POS categories (nouns, proper nouns, verbs, adjectives) is meant to spend more of the vocabulary budget on complete content words. That results in various benefits and drawbacks, depending on the language. First, it decreases the percentage of cases in which a token spans a boundary in all languages (Table 9), while in Tamil for example the recall increases from 7.4 to 32.7 (plain BPE hardly cuts at the stem boundary). In Spanish and Turkish however it decreases recall because whole-word tokens take the place of suffix tokens that plain BPE learns. The weight of 2 was not tuned; varying it would make it a tuned method, which we did not do.

5.4 Lemma2Word

Table 3 distinguishes between the variant that uses only learned boundaries (the lexicon variant) and the variant that lemmatizes each word on the fly using Stanza (the online variant). All forms, forms not in training text, and UD test split forms are also reported separately.

The lexicon variant has a major improvement over the unweighted BPE in recall, from 54.4 to 88.2 percent in English and 7.4 to 84.5 percent in Tamil, and 23–34 percentage points for the

[1]Seed 0; the three-seed mean of the share of tokens that span a boundary falls from 22.0% to 15.6% (Table 9).

Table 3: Boundary recall and precision (%) in Lemma2Word, by subset of words and variant. The lexicon variant uses only learned boundaries and affixes, the online variant lemmatizes the word to be tokenized with Stanza. Results averaged over three random seeds.

		all		unseen		UD test	
Language	Variant	R	P	R	P	R	P
English	lexicon	88.2	45.3	73.7	30.3	84.0	37.2
	online	98.2	50.9	98.2	41.1	91.1	40.9
	BPE, free marker	54.4	31.6	54.5	25.5	49.2	24.8
Spanish	lexicon	62.2	25.6	40.2	15.3	48.1	18.3
	online	99.1	39.0	99.1	35.4	94.8	34.2
	BPE, free marker	29.4	13.7	35.8	14.7	31.8	13.2
Danish	lexicon	70.8	31.7	49.5	17.9	65.5	27.2
	online	97.2	43.6	97.1	35.3	88.0	37.0
	BPE, free marker	37.9	17.9	41.0	15.9	38.0	16.7
Tamil	lexicon	84.5	38.5	74.2	27.4	80.6	34.7
	online	97.6	46.3	97.7	38.5	91.5	39.9
	BPE, free marker	7.4	3.2	7.0	2.5	5.9	2.4
Turkish	lexicon	83.3	34.4	81.7	31.2	82.7	32.0
	online	88.6	37.6	88.4	34.8	89.1	35.6
	BPE, free marker	60.2	24.7	60.5	23.2	60.4	23.2

other three languages. Precision is improved as well, since BPE on the shorter pieces learns fewer internal boundaries. The 20-occurrence threshold also admits fewer of the short and ambiguous inflectional suffixes of Spanish and Danish in combinations of word-final letter sequences. In terms of unseen forms, recall is 73.7 percent in English, 74.2 in Tamil, and falls to a few points in Spanish and Danish for the short and ambiguous word-final letter sequences.

The online variant recovers nearly all the correct boundaries, reaching 99.1 percent recall in Spanish because the gold boundaries were extracted from the lemmas, and the lemmatizer could recall the correct lemma from its memory on the training data. On the UD test-split forms, which the lemmatizer never saw, recall drops, but is still within 88.0–94.8 percent. Its precision, however, is no better than 51 percent, because the BPE can still split inside the input pieces, while the gold standard marks only the boundaries around the word stem.

5.5 Compression and Stability

Table 4 reports the average tokenization efficiency, measured as the average number of tokens produced per word, on the held-out text. The effect of the POS weighting on compression is less than one percent. In English, Lemma2Word produces 1.538 tokens per word compared to 1.481 for the free-marker BPE, an increase of four percent. In Spanish the increase is larger, eight percent, with Lemma2Word producing 1.531 tokens per word compared to 1.413 for the free-marker BPE. Lemma2Word is more compact than the unigram model for all languages except Tamil. The tokenizers that are provided as reference never cross a word boundary, and are more expensive in terms of tokens per word in the three European languages, and cheaper in the two others (Tamil, Turkish), because the free-marker BPE budget in these two languages is partly spent on tokens that cover frequent word pairs.

The reliability is measured in the appendices (Tables 10 and 11) by comparing the vocabularies produced on different training samples, and the segmentations of the same held-out text produced

Table 4: Tokens per whitespace word on held-out Wikipedia paragraphs (lower is more compact).

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	1.481 ±0.002	1.413 ±0.002	1.606 ±0.002	2.242 ±0.005	1.962 ±0.002
POS-weighted BPE	1.470 ±0.001	**1.412 ±0.001**	**1.604 ±0.002**	2.215 ±0.006	1.938 ±0.025
transitions only	1.470 ±0.001	1.412 ±0.001	1.604 ±0.002	2.215 ±0.006	1.938 ±0.025
penalty only	**1.470 ±0.001**	1.413 ±0.002	1.605 ±0.002	2.243 ±0.005	1.962 ±0.003
Content-word-weighted BPE	1.486 ±0.002	1.423 ±0.002	1.619 ±0.001	2.150 ±0.012	1.998 ±0.003
Lemma2Word + BPE	1.538 ±0.001	1.531 ±0.002	1.703 ±0.001	2.376 ±0.002	2.080 ±0.010
BPE, word-initial marker	1.555 ±0.001	1.565 ±0.000	1.713 ±0.001	**2.042 ±0.001**	**1.883 ±0.002**
Unigram LM	1.609 ±0.001	1.607 ±0.002	1.778 ±0.004	2.177 ±0.006	2.004 ±0.004

by different models trained on different samples using the same method. In both cases, the reliability statistics are far less discriminative than accuracy. The Jaccard measure of the vocabularies shows a range of 62–79 percent overlap in entries for vocabularies produced from different training samples; the segmentations produced by the same method show a range of 88–97 percent agreement. The POS weighting does not improve these statistics, and for the overlap in the Turkish vocabulary the POS weighting is actually a detriment, going from 74.6 to 67.1 percent, with a standard deviation of 7.4 percentage points, the least stable of the cases shown in the table. The unigram model has the lowest vocabulary overlap in all languages and the same range of agreement as all the other methods, and the difference in the vocabularies produced from the unigram model is restricted mostly to rare entries that are unlikely to be seen in the held-out text.

5.6 Vocabulary Budget

Comparing the performance of the different methods at two other vocabulary budgets (2,000 and 32,000 types) in Figure 3, the relative ranking of the methods is maintained at all budgets. Lemma2Word improves with larger vocabularies, moving up from 48.9 to 68.1 percent F1 in English, for example, due to the larger budget allowing more word pieces to remain unsplit. The unigram model moves down at 32,000 types in English, Tamil, and Turkish, where the budget exceeds what a 3,000-paragraph sample can support and whole words enter the vocabulary. The difference between the POS-weighted BPE and the unweighted BPE is less than three percentage points at all budget sizes.

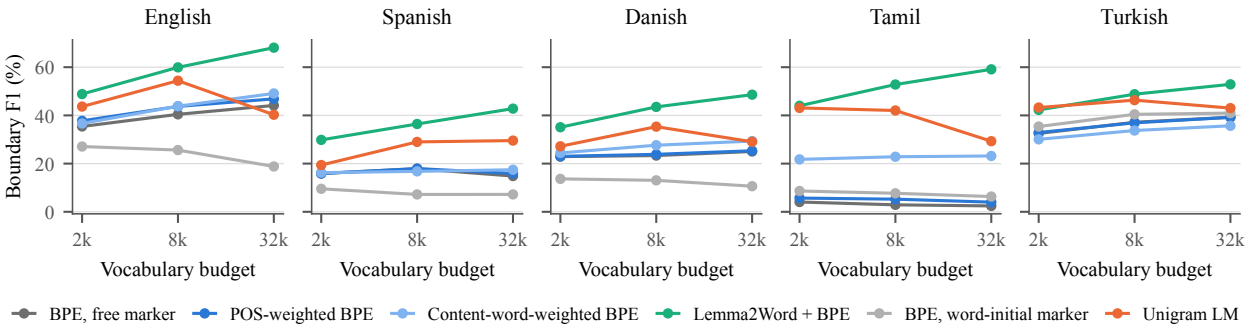


Figure 3: Boundary F1 (%) at vocabulary budgets of 2,000, 8,000, and 32,000 types using seed 0.

5.7 Language-Model Probe

Table 5 reports the bits per character of the language model trained on the different tokenizers, and Figure 4 shows the same data graphically. A language model is trained for each tokenizer seed, and across these different seeds the standard deviation is less than 0.01 bits in most cases, and never more than 0.03 bits. Differences of a few hundredths of a bit can therefore be considered reliable at this scale.

Table 5: Bits per character in the language model trained on the output of the tokenizer. The language model has 7M parameters and is trained for 6 passes over the same text with each tokenizer. Lower scores are better, and we show the mean and one standard deviation for the three tokenizer seeds.

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	2.807 ±0.002	2.689 ±0.003	2.878 ±0.001	2.848 ±0.006	2.963 ±0.002
POS-weighted BPE	2.785 ±0.001	2.687 ±0.001	2.874 ±0.001	2.819 ±0.005	2.935 ±0.032
transitions only	2.785 ±0.001	2.687 ±0.001	2.874 ±0.002	2.819 ±0.005	2.935 ±0.032
penalty only	2.786 ±0.002	2.689 ±0.002	2.876 ±0.001	2.849 ±0.006	2.963 ±0.003
Content-word-weighted BPE	2.794 ±0.001	2.680 ±0.003	2.868 ±0.001	2.737 ±0.015	2.995 ±0.003
Lemma2Word + BPE	2.887 ±0.003	2.861 ±0.004	3.007 ±0.001	2.983 ±0.002	3.096 ±0.011
BPE, word-initial marker	2.715 ±0.003	2.674 ±0.006	2.909 ±0.003	2.611 ±0.001	2.808 ±0.004
Unigram LM	**2.531** ±0.006	**2.473** ±0.011	**2.704** ±0.007	**2.524** ±0.003	**2.694** ±0.003

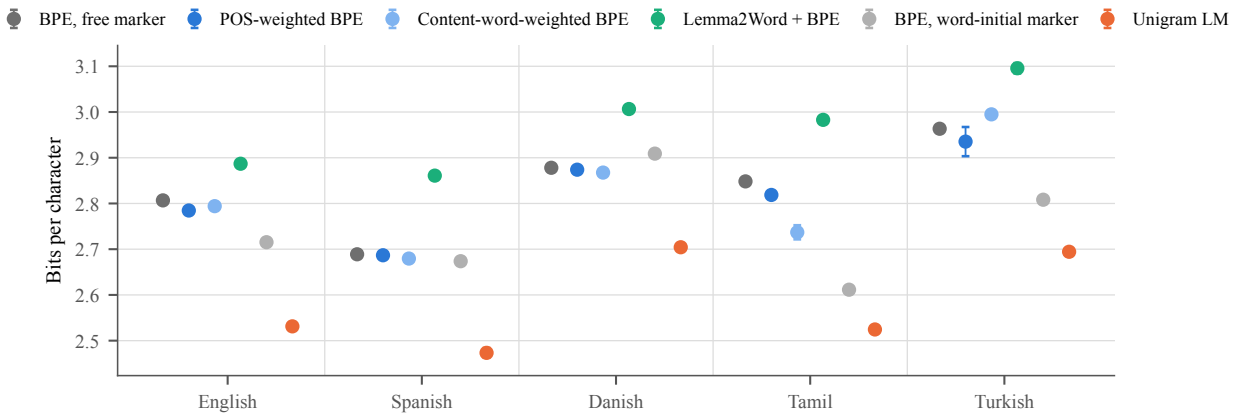


Figure 4: Bits per character in the language model trained on the output of the tokenizer. The language model has 7M parameters and is trained for 6 passes over the same text with each tokenizer. Lower scores are better, and we show the mean and one standard deviation for the three tokenizer seeds. The y-axis starts at 2.4 bits to show the differences.

However, the ordering by bits-per-character does not match the ordering by boundary-F1. The unigram model has been ranked second by F1 and first by bits-per-character in all languages. In English it is first with 2.531 bits-per-character compared to 2.807 for free-marker BPE. In Tamil the first place is taken by the unigram model with 2.524 bits compared to 2.848 for free-marker BPE. Lemma2Word has been ranked first by F1 and last by bits-per-character in all languages. In English Lemma2Word is ranked last with 2.887 bits-per-character. The word cutting using stem boundaries before training the subword tokenizer is removing frequent full-word tokens and adding tokens that are too unpredictable for a model of this size to compensate.

Word-initial BPE has been ranked last by F1 in four languages. In four languages, it has been ranked ahead of free-marker BPE by bits-per-character. In English it is ranked ahead with 2.715

vs. 2.807. The tokens crossing word boundaries, which help compactness in free-marker BPE, do not help in prediction. POS weighting has been changing bits-per-character by less than 0.03 in all the languages, so it seems it is pointing in a positive direction but with a small contribution. Interestingly, the table shows that for Turkish the seed-to-seed variation in bits-per-character is the highest. The two ablations with POS weighting give the same result. Content word weighting has been reducing bits-per-character in Tamil to 2.737 vs. 2.848. This was also the language where it has been giving the largest improvement in boundary-F1. On the other hand, in Turkish content-word weighting has been increasing the bits-per-character numbers.

Since all the tokenizers are trained on the same text, the number of passes is the same for all models; they go over the data 6 times. Since a higher token count therefore leads to more optimizer steps, the unigram model has 8.6% more optimizer steps than free-marker BPE in English. To control for the change in optimizer steps, we have trained a model for free-marker BPE in seed 0 with 10% fewer optimizer steps (5.4 passes over the data) and 10% more optimizer steps (6.6 passes). The bits-per-character changes from 2.807 to 2.808 and 2.805 respectively. These changes are small compared to the ones between tokenizers, so the step count cannot be used to explain the model ranking. Since Lemma2Word has a higher number of tokens than free-marker BPE and it has been ranked behind, the reason cannot be the optimizer step count alone.

These results indicate that morphological alignment and the quality of the model do not move together at this scale, which agrees with the conclusions reached about the unigram model by Bostrom and Durrett [6]. This also supports the finding that the intrinsic measures for tokenizers are not perfect predictors for downstream tasks [1, 22].

5.8 A Language Model as a Segmenter

Table 6 shows the results of the study described in Section 3.6. For each language, a sample of 400 word forms were randomly selected from the gold standard list using seed 0. The Claude Sonnet model (`claude-sonnet-5-5`, October 2026 version) was selected and it was prompted to add plus signs between morphemes in batches of 40 words. The prompt used does not allow any character modification and is given in Appendix C. Answers that change the characters of the word are invalid and they were not counted when computing the scores. To compare with the same seed 0 tokenizers, we have evaluated them on the same words that the model had given valid answers for.

Table 6: Claude Sonnet segmenting words into morphemes, on a fixed random sample of gold standard forms. The seed 0 tokenizers are evaluated on the same words. Invalid answers are answers that change the characters of the word and they were not counted in the computation of the scores. The table reports boundary precision, recall and F1 (in %), and the number of tokens per word after segmentation.

			language model				recall of tokenizers on the same words		
Language	words	invalid	P	R	F1	pieces/word	Lemma2Word	Unigram	BPE, free
English	400	0	77.6	91.0	83.8	2.17	87.0	69.2	52.2
Spanish	399	1	35.1	69.9	46.8	2.99	62.2	44.1	25.3
Danish	400	0	38.9	66.0	49.0	2.69	72.5	50.0	42.0
Tamil	393	7	72.6	90.4	80.5	2.25	84.8	48.9	6.8
Turkish	400	0	50.7	96.5	66.5	2.90	84.8	67.2	60.2

The model has been able to surpass the recall on gold boundaries of any of the tokenizers in four languages. In English the model gets 91.0% compared to 87.0% for Lemma2Word, in Tamil 90.4% vs. 84.8% and in Turkish 96.5% vs. 84.8%. The model has been ranked behind Lemma2Word in Danish with 66.0% recall vs. 72.5%. The precision of the model is ranging between 35.1% and

77.6%. The model marks derivational affixes and parts of compounds that are not included in the lemma-based gold standard, and the number of pieces per word is between 2.17 and 2.99 compared to the maximum of 3 pieces per word in the gold. Almost all of the answers preserve the characters of the words in the Latin script, and there was only 1 invalid answer for Spanish and none for the other Latin-script languages.

Tamil had two issues. The model had long reasoning chains, the 40-word batches exceeded the response length, and the unanswered words had to be re-asked in 15-word batches. The model had also written the underlying morphemes in 7 answers rather than the surface forms, violating the character constraint. In one of them the model gave கள் + உக்கு instead of களுக்கு, since the suffix vowel fuses with the consonant. The study used approximately 2,000 words and the cost of the API was minimal (a few dollars). This evaluation is not showing what a tokenizer built on top of the model's responses would be capable of, but what the model knows about the morphology. The answers from the model do not have a fixed vocabulary and they are not guaranteed to be in a consistent format even if the same word occurs multiple times. The cost per word is several orders of magnitude more than any of the tokenizers shown in Table 1. The answers could be used off-line to generate a segmentation lexicon similar to the output of Lemma2Word to provide boundaries in places where a lemmatizer doesn't exist. This use case is not evaluated in the paper.

6 Discussion

The transition-weighting fails because it adds no new signal. The basic BPE merge score is just the count of a pair of adjacent symbols. The variant that uses POS weighting simply multiplies this count by one plus the probability of the corresponding transition in the tag sequence, when the pair of symbols is spanning the boundary between two words. This is a smoothing of the original symbol-pair count, because the transition probability is simply a function of the pair of tags and the tags are functions of the words. In the case of the pair *of the*, the high frequency results in a merge, and the weighting just adds incentive towards this frequent pair.

As Table 2 shows, the same tokens bridge the word boundary with and without weighting. There is no signal pointing to the right place to split at a token boundary, since the probability of a transition at a boundary spanned by a token is never greater than the probability of a transition at an unspanned boundary. The design simply never intended to signal the right place to split, which is what is needed to find the gold boundaries in the middle of words.

In the preliminary results reported earlier, the segmentation entropy was lower in the tokenization using POS tagging, the tokens were longer, the transition accuracy diagnostic showed gains of 12–20 points, and vocabulary reuse showed gains of about 20 points. These were not measured on held-out words, or against a gold standard. The implementation used was the same as that with the bug in Section 3.2 of comparing the transitions against the wrong words after the first space-absorbing merge. Under the protocol in Section 4, there was no change in vocabulary reuse (Table 10), and less than 1% change in the length of the tokens (Table 4). And the only method that consistently improved the detection of morphological boundaries is the one using a lemmatizer.

The lemma carries information about the end of the stem, whereas the tag classifies the type of word, and only the lemma contributes information about the boundary. The Lemma2Word method has 23 to 77 point gains in recall, of which the proportion generalizing to out-of-vocabulary words depends on the generalization in the inventories of the affixes. This is good in English and Turkish, where the suffixes are long and distinct, and bad in Spanish and Danish, where the suffixes are short. The online version generalizes through the lemmatizer and reaches 88%–95% recall on the forms in the treebank test set, but requires a neural lemmatizer to run online during the tokenization and

uses the same training data as the gold standard.

The content-word weighting sits between the two. It uses the tags to make the boundary decision by choosing the words that are preserved as whole words. It helps in the case where the budget was missing whole words, as in the case of content words in Tamil, but hurts in the case where the budget could be better spent on suffixes, as in Spanish and Turkish.

The probe in Section 5.7 separates two things that the literature on morphology-aware tokenization often conflates. The probe shows that more gold boundaries does not mean better downstream performance for the language model. The tokenizer that finds the most boundaries, Lemma2Word, is the one that corresponds to the worst model performance. The unigram model, which finds fewer boundaries but segments text according to a probabilistic model, is the one that corresponds to the best performance, by about a quarter of a bit per character. The probe is small in scale, and a larger model using more text may change or invert these differences. However, this direction is consistent with Bostrom and Durrett [6], who found that unigram segmentation performs at least as well as BPE in pretraining, and the results by Ali et al. [1] and Schmidt et al. [22], who find that performance on intrinsic measures of tokenizers is poorly predictive of performance on downstream tasks. The importance of the result is that a score on the morphological boundaries is not in itself a sufficient justification to use a tokenizer.

The space marker convention is another factor that is rarely considered. The original implementation is such that the space marker is freely merged. In this implementation, 4%–22% of the tokens span a word boundary. This increases the boundary F1 over the word-initial marker convention in four languages. This is because the whole words with a space marker prefixed are not single tokens. This means that the budget of whole words is spent on smaller pieces. This however disagrees with the probe, which finds a preference for the word-initial marker convention in the same four languages. This is as important as the linguistic signals chosen. The comparisons between different tokenizers should maintain a fixed convention for the space marker.

There are limitations in this study. The gold standard is limited to two boundaries per word, which are around the stem. This means that the internal structure of the derivational morphology is counted against precision. This makes recall the more reliable metric, although the ordering of the methods is unchanged. The training samples are small with 3,000 paragraphs from each language. This is in part because the implementation is pure Python for the free space marker convention. The sweeps in vocabulary size show that the order of the methods is the same at each budget, but the absolute numbers can change in larger corpora. The tags and lemmas are from the same tagger. The Tamil treebank is the smallest of the five treebanks used. The language model probe uses a small model with 7M parameters that was trained for minutes only. This is able to show tokenizer differences at the scale of this probe, but not at the scale of a large language model.

7 Conclusion

Using part-of-speech (POS) and lemma information in subword tokenizers in five languages, we assess them under one protocol: gold morpheme boundaries, held-out text, repeated training samples and a language model probe. Weighting subword merges by POS transitions does not change token boundaries or output, because the transition probability only describes the order of words, not internal word structure. Weighting subword merges within content words changes the vocabulary as desired, but does not always give a better accuracy than without.

Using lemmas to deduce word boundaries instead increases boundary recall by 23 to 77 points, at the cost of 4–8% more tokens. How much of the gain carries over to unseen words depends on the distinctiveness of a language's affixes. If a tokenizer is to include morphological boundaries,

it should use a lemmatizer or segmentation lexicon, where a tagger does not provide any value. However, the quality of these boundaries for language models is a question in itself. A language model probe ranks these tokenizers differently than the boundary scores, with the unigram model first and Lemma2Word last in each language, and will need larger scale than our 7M-parameter language model to resolve. Morphological alignment cannot yet be recommended as a goal.

Author contributions. A.C., J.D. and M.V. designed the POS-weighted tokenizer, Lemma2Word and the language model segmentation study and wrote the original report. J.D. re-implemented, corrected and evaluated the tokenizers, ran the experiments and wrote the current report.

References

[1] Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Buschhoff, Charvi Jain, Alexander Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, Stefan Kesselheim, and Nicolas Flores-Herr. Tokenizer choice for LLM training: Negligible or crucial? In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3907–3924, Mexico City, Mexico, 2024. Association for Computational Linguistics.

[2] Catherine Arnett and Benjamin Bergen. Why do language models perform worse for morphologically complex languages? In *Proceedings of the 31st International Conference on Computational Linguistics*, Abu Dhabi, UAE, 2025. Association for Computational Linguistics.

[3] Catherine Arnett, Marisa Hudspeth, and Brendan O'Connor. Evaluating morphological alignment of tokenizers in 70 languages. *arXiv preprint arXiv:2507.06378*, 2025.

[4] Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies. *arXiv preprint arXiv:2502.00894*, 2025.

[5] Khuyagbaatar Batsuren, Ekaterina Vylomova, Verna Dankers, Tsetsuukhei Delgerbaatar, Omri Uzan, Yuval Pinter, and Gábor Bella. Evaluating subword tokenization: Alien subword composition and OOV generalization challenge. *arXiv preprint arXiv:2404.13292*, 2024.

[6] Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4617–4624. Association for Computational Linguistics, 2020.

[7] Maharaj Brahma, Baby Pavan, Pavan Gopinath, Kartik Nigam, Hamza Shaikh, and Ashutosh Modi. MorphTok: Morphologically grounded tokenization for Indian languages. *arXiv preprint arXiv:2504.10335*, 2025.

[8] Mathias Creutz and Krista Lagus. Unsupervised models for morpheme segmentation and morphology learning. *ACM Transactions on Speech and Language Processing*, 4(1):1–34, 2007.

[9] Matthias Gallé. Investigating the effectiveness of BPE: The power of shorter sequences. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1375–1381, Hong Kong, China, 2019. Association for Computational Linguistics.

[10] Valentin Hofmann, Janet Pierrehumbert, and Hinrich Schütze. Superbizarre is not superb:

Derivational morphology improves BERT's interpretation of complex words. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3594–3608. Association for Computational Linguistics, 2021.

[11] Valentin Hofmann, Hinrich Schütze, and Janet Pierrehumbert. An embarrassingly simple method to mitigate undesirable properties of pretrained language model tokenizers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 385–393, Dublin, Ireland, 2022. Association for Computational Linguistics.

[12] Haris Jabbar. MorphPiece: A linguistic tokenizer for large language models. *arXiv preprint arXiv:2307.07262*, 2023.

[13] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, 2018. Association for Computational Linguistics.

[14] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, 2018. Association for Computational Linguistics.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[16] Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

[17] Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Jan Hajič, Christopher D. Manning, Sampo Pyysalo, Sebastian Schuster, Francis Tyers, and Daniel Zeman. Universal dependencies v2: An evergrowing multilingual treebank collection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4034–4043, Marseille, France, 2020. European Language Resources Association.

[18] Antoine Nzeyimana and Andre Niyongabo Rubungo. KinyaBERT: a morphology-aware Kinyarwanda language model. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5347–5363, Dublin, Ireland, 2022. Association for Computational Linguistics.

[19] Hyunji Hayley Park, Katherine J. Zhang, Coleman Haley, Kenneth Steimel, Han Liu, and Lane Schwartz. Morphology matters: A multilingual language modeling analysis. *Transactions of the Association for Computational Linguistics*, 9:261–276, 2021.

[20] Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. Stanza: A Python natural language processing toolkit for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 101–108. Association for Computational Linguistics, 2020.

[21] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? On the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long*

Papers), pages 3118–3135. Association for Computational Linguistics, 2021.

[22] Craig W. Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. Tokenization is more than compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 678–702, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[23] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics.

[24] Cagri Toraman, Eyup Halit Yilmaz, Furkan Şahinuç, and Oguzhan Ozcelik. Impact of tokenization on language models: An analysis for Turkish. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 22(4), 2023.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.

[26] Wikimedia Foundation. Wikipedia dumps, snapshot of 1 november 2023, as the Hugging Face dataset `wikimedia/wikipedia`. `https://huggingface.co/datasets/wikimedia/wikipedia`, 2023.

[27] Yue Zhang and Stephen Clark. Joint word segmentation and POS tagging using a single perceptron. In *Proceedings of ACL-08: HLT*, pages 888–896, Columbus, Ohio, 2008. Association for Computational Linguistics.

[28] Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5184–5207, Toronto, Canada, 2023. Association for Computational Linguistics.

A Precision, Recall and Stability by Language

Table 7: Boundary precision (%).

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	31.6 ±1.2	13.7 ±0.5	17.9 ±0.6	3.2 ±1.7	24.7 ±1.4
POS-weighted BPE	33.6 ±1.1	13.8 ±0.6	18.1 ±0.3	4.5 ±1.7	24.6 ±1.3
transitions only	33.6 ±1.2	13.8 ±0.6	18.1 ±0.3	4.6 ±1.7	24.6 ±1.3
penalty only	32.7 ±0.9	13.7 ±0.5	18.0 ±0.7	3.2 ±1.7	24.6 ±1.5
Content-word-weighted BPE	34.3 ±0.2	12.1 ±0.2	20.1 ±0.5	16.8 ±0.9	24.2 ±0.3
Lemma2Word + BPE	**45.3 ±0.1**	**25.6 ±0.1**	**31.7 ±0.2**	**38.5 ±0.4**	34.4 ±0.1
BPE, word-initial marker	22.0 ±0.3	5.3 ±0.4	10.2 ±0.1	6.4 ±0.3	31.0 ±0.3
Unigram LM	45.0 ±0.5	21.9 ±0.0	27.2 ±0.4	36.9 ±0.8	**35.0 ±0.3**

Table 8: Boundary recall (%).

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	54.4 ±2.6	29.4 ±1.5	37.9 ±1.5	7.4 ±3.7	60.2 ±2.4
POS-weighted BPE	61.3 ±2.5	29.8 ±1.5	37.9 ±1.3	9.5 ±3.3	60.7 ±2.1
transitions only	61.3 ±2.5	29.8 ±1.5	37.9 ±1.3	9.6 ±3.3	60.7 ±2.1
penalty only	58.7 ±2.0	29.4 ±1.5	37.9 ±1.6	7.4 ±3.7	60.1 ±2.5
Content-word-weighted BPE	60.8 ±0.3	25.9 ±0.5	44.8 ±1.1	32.7 ±1.6	57.8 ±0.4
Lemma2Word + BPE	**88.2 ±0.1**	**62.2 ±0.3**	**70.8 ±0.5**	**84.5 ±0.5**	**83.3 ±0.3**
BPE, word-initial marker	30.3 ±0.4	9.1 ±0.7	18.4 ±0.2	9.0 ±0.4	59.6 ±0.7
Unigram LM	68.6 ±0.4	42.9 ±0.2	49.3 ±0.6	51.5 ±0.9	69.3 ±0.7

Table 9: Share of held-out tokens that span a word boundary (%).

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	15.2 ±0.1	18.3 ±0.0	13.1 ±0.1	**22.0 ±1.5**	4.5 ±0.1
POS-weighted BPE	14.2 ±0.1	18.4 ±0.0	13.2 ±0.2	15.6 ±2.4	4.4 ±0.1
transitions only	14.2 ±0.1	18.4 ±0.0	13.2 ±0.2	15.6 ±2.4	4.4 ±0.1
penalty only	14.4 ±0.0	18.3 ±0.0	13.2 ±0.1	21.9 ±1.5	4.5 ±0.1
Content-word-weighted BPE	10.9 ±0.2	14.5 ±0.1	9.7 ±0.2	2.2 ±0.3	2.5 ±0.1
Lemma2Word + BPE	**15.7 ±0.0**	**19.2 ±0.0**	**15.4 ±0.1**	12.8 ±0.2	**6.6 ±0.1**
BPE, word-initial marker	0.0 ±0.0	0.0 ±0.0	0.0 ±0.0	0.0 ±0.0	0.0 ±0.0
Unigram LM	0.0 ±0.0	0.0 ±0.0	0.0 ±0.0	0.0 ±0.0	0.0 ±0.0

Table 10: Vocabulary reuse: Jaccard overlap (%) between the vocabularies learned from different training samples (three seed pairs).

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	76.8 ±0.8	75.8 ±1.8	76.8 ±0.4	63.8 ±4.8	74.6 ±1.9
POS-weighted BPE	77.8 ±0.9	76.3 ±2.0	76.7 ±0.6	61.9 ±5.4	67.1 ±7.4
transitions only	77.8 ±0.9	76.2 ±2.0	76.6 ±0.5	61.8 ±5.4	67.2 ±7.4
penalty only	77.1 ±0.7	75.8 ±1.8	76.3 ±0.4	64.1 ±4.8	75.1 ±2.3
Content-word-weighted BPE	74.8 ±3.1	**77.2 ±0.3**	74.3 ±2.4	71.5 ±2.5	77.0 ±1.0
Lemma2Word + BPE	76.9 ±0.7	75.6 ±1.5	**77.8 ±0.6**	74.8 ±0.5	74.1 ±3.1
BPE, word-initial marker	**78.5 ±0.4**	74.4 ±3.2	76.1 ±0.9	**76.9 ±0.2**	**78.5 ±1.1**
Unigram LM	69.3 ±1.1	66.3 ±0.7	62.7 ±0.2	62.9 ±0.6	69.1 ±0.4

B Example Segmentations

C Prompt of the Segmentation Study

The language model was prompted with the following text string, which contained the name of the respective language and a batch of words, one per line.

```
Segment each {lang} word below into its morphemes: prefixes, the stem, and
suffixes (inflectional and derivational).
Rules: keep every character of the word exactly as given and in order; insert a
plus sign (+) at each morpheme boundary; a word with a single morpheme is
returned unchanged; do not add explanations.
Answer with one line per word in the format   word: seg+men+ted   and nothing
```

Table 11: Segmentation agreement: boundary F1 (%) between the held-out segmentations of tokenizers trained on different samples (three seed pairs).

System	English	Spanish	Danish	Tamil	Turkish
BPE, free marker	95.2 ±0.1	95.1 ±0.8	95.7 ±0.2	88.6 ±3.4	94.6 ±1.2
POS-weighted BPE	95.8 ±0.1	95.2 ±0.9	95.5 ±0.3	87.5 ±3.3	92.5 ±2.7
transitions only	95.8 ±0.1	95.3 ±0.9	95.5 ±0.3	87.6 ±3.4	92.5 ±2.7
penalty only	95.6 ±0.2	95.1 ±0.8	95.5 ±0.1	88.7 ±3.5	94.7 ±1.3
Content-word-weighted BPE	95.0 ±1.1	**95.9 ±0.1**	95.5 ±0.6	94.0 ±0.9	**96.0 ±0.2**
Lemma2Word + BPE	95.2 ±0.3	94.9 ±0.6	95.8 ±0.2	94.7 ±0.2	95.6 ±0.7
BPE, word-initial marker	**96.9 ±0.1**	95.5 ±1.2	**95.9 ±0.1**	**95.4 ±0.2**	95.8 ±0.6
Unigram LM	95.0 ±0.1	94.1 ±0.2	93.5 ±0.2	93.5 ±0.1	94.3 ±0.2

Table 12: Example segmentations of evaluation words absent from the training text (seed 0). The evaluation words are taken from unseen forms without capitals, have between 8 and 14 characters and at least one gold boundary. We show the first, middle and last word of an alphabetically sorted list. To save space, the word-initial-marker BPE is omitted. Vertical bars (|) indicate boundaries, and gold contains the ground-truth boundaries inferred from the lemma.

Lang.	Gold	BPE, free marker	Content-weighted	Lemma2Word	Unigram
En	abduct|ed	ab|duct|ed	ab|duct|ed	ab|duc|t|ed	a|b|du|c|ted
En	interrupt|ing	inter|rupt|ing	inter|rupt|ing	inter|rup|t|ing	interrupt|ing
En	youngster|s	young|st|ers	young|st|ers	young|ster|s	young|ster|s
Sp	abandonado|s	abandon|ados	abandon|ados	abandon|a|dos	abandonado|s
Sp	galeote|s	gal|e|ot|es	gal|e|ot|es	gal|e|ot|es	gal|eo|tes
Sp	ímprobo|s	í|m|prob|os	í|m|prob|os	í|m|prob|os	í|mp|ro|bo|s
Da	abonnent|erne	ab|on|n|ent|erne	abon|n|ent|ern|e	ab|on|n|ent|erne	ab|on|n|en|t|erne
Da	landshold|ets	lands|hol|det|s	lands|hold|et|s	lands|hold|et|s	lands|hold|ets
Da	østmagt|ernes	øst|magt|ern|es	øst|magt|ern|es	øst|magt|erne|s	øst|magt|ernes
Ta	ஹெர்மனி|யில்	g|எ|ர்மன|ிய|ில்	g|எ|ர்மன|ிய|ில்	g|எ|ர்ம|னி|யில்	g|எ|ர்|ம|னி|யில்
Ta	தற்கொலை|ப்	தற்க|ொலைப|்	தற்கொ|லை|ப்	தற்க|ொ|ல|ை|ப்	தற்கொலை|ப்
Ta	ஹென்றி|யின்	ஹ|ென்ற|ிய|ின்	ஹெ|ன்ற|ிய|ின்	ஹெ|ன்ற|ி|ய|ின்	ஹென்றி|யின்
Tu	abajur|lara	ab|aj|ur|lar|a	ab|aj|ur|lar|a	aba|j|ur|lar|a	a|ba|j|ur|lara
Tu	korkala|mışt ı	kor|kal|amış|t ı	kor|kal|amış|t|ı	kor|kal|amış|t ı	kor|ka|la|mışt ı
Tu	şıpıdık|larının	ş|ıp|ı|dık|ların|ın	ş|ıp|ıd|ık|larının	ş|ıp|ı|dık|ların|ın	ş|ıp|ı|dıkları|nın

```
else.

{words}
```

D Reproduction

The repository contains all the code and data preparation scripts, as well as the metrics for each run. We present the commands here to reproduce all the numbers, tables and figures.

1. `python experiments/fetch_corpus.py` fetches the corpus from Wikipedia paragraphs.
2. `bash experiments/tag_all.sh` runs the tagging and lemma tasks using Stanza.
3. `bash experiments/run_all.sh` runs the main experiments, over five languages, eight methods, and three seeds.
4. `bash experiments/run_sweep.sh` runs the sweep experiments with two type counts (2,000 and 32,000) and seed 0.

5. `bash experiments/after_grid.sh` runs the language model probes.
6. `python experiments/llm_segment.py` runs the large language model segmentation study.
7. `python experiments/aggregate.py` produces `numbers.tex`, tables, and figures.

Gold boundaries come from the Hugging Face dataset `catherinearnett/morphscore`, and text from the Hugging Face dataset `wikimedia/wikipedia`.